

SIMULATION TASK

```
clc;
clear;
close;

// RATIONAL SAMPLING RATE CONVERSION
// Input Fs = 48 kHz
// Output Fs = 32 kHz
// L = 2, M = 3
Fs_in = 48000;
Fs_out = 32000;
L = 2;
M = 3;

// 1. Generate input signal
T = 0.02;           // Signal duration
t = 0:1/Fs_in:T-1/Fs_in;
f1 = 5000;          // 5 kHz
f2 = 10000;         // 10 kHz
x = sin(2*pi*f1*t) + ...
    0.5*sin(2*pi*f2*t);

// 2. Plot input signal
scf(1);
plot(t(1:500),x(1:500));
xlabel("Time (seconds)");
ylabel("Amplitude");
title("Input Signal - 48 kHz");
xgrid();
```

```

// 3. Spectrum before conversion
N = length(x);
X = fft(x);
f = (0:N-1)*(Fs_in/N);
scf(2);
plot(f(1:floor(N/2)), ...
     abs(X(1:floor(N/2)))/N);
xlabel("Frequency (Hz)");
ylabel("Magnitude");
title("Spectrum Before Conversion");
xgrid();

// 4. INTERPOLATION BY L = 2
// Insert one zero between consecutive samples
x_interp = zeros(1,L*length(x));
for k = 1:length(x)
    x_interp((k-1)*L+1) = x(k);
end
Fs_interp = Fs_in * L;
fprintf("\nAfter interpolation = %.2f kHz\n", ...
       Fs_interp/1000);

// 5. Low-pass filtering
// Simple moving-average FIR filter
// Used to remove spectral images
h = [1 2 3 4 5 4 3 2 1];
h = h/sum(h);
x_filtered = conv(x_interp,h);

```

```

// Remove filter delay approximately
delay = floor(length(h)/2);
x_filtered = x_filtered(delay+1:delay+length(x_interp));
// 6. DECIMATION BY M = 3
x_out = x_filtered(1:M:length(x_filtered));
Fs_final = Fs_interp/M;
mprintf("After decimation = %.2f kHz\n", ...
        Fs_final/1000);
// 7. Plot output signal
t_out = 0:1/Fs_final:(length(x_out)-1)/Fs_final;
scf(3);
plot(t_out(1:min(500,length(t_out))), ...
      x_out(1:min(500,length(x_out))));
xlabel("Time (seconds)");
ylabel("Amplitude");
title("Output Signal - 32 kHz");
xgrid();
Nout = length(x_out);
Xout = fft(x_out);
fout = (0:Nout-1)*(Fs_final/Nout);
scf(4);
plot(fout(1:floor(Nout/2)), ...
      abs(Xout(1:floor(Nout/2)))/Nout);
xlabel("Frequency (Hz)");
ylabel("Magnitude");
title("Spectrum After Conversion - 32 kHz");
xgrid();

```

```

// 9. Compare spectra
scf(5);
subplot(2,1,1);
plot(f(1:floor(N/2)), ...
      abs(X(1:floor(N/2)))/N);
xlabel("Frequency (Hz)");
ylabel("Magnitude");
title("Before Conversion - 48 kHz");
xgrid();
subplot(2,1,2);
plot(fout(1:floor(Nout/2)), ...
      abs(Xout(1:floor(Nout/2)))/Nout);
xlabel("Frequency (Hz)");
ylabel("Magnitude");
title("After Conversion - 32 kHz");
xgrid();

//
=====

// 10. Verify sampling rate conversion
//
=====

mprintf("\n=====\\n");
mprintf("SAMPLING RATE CONVERSION\\n");
mprintf("=====\\n");

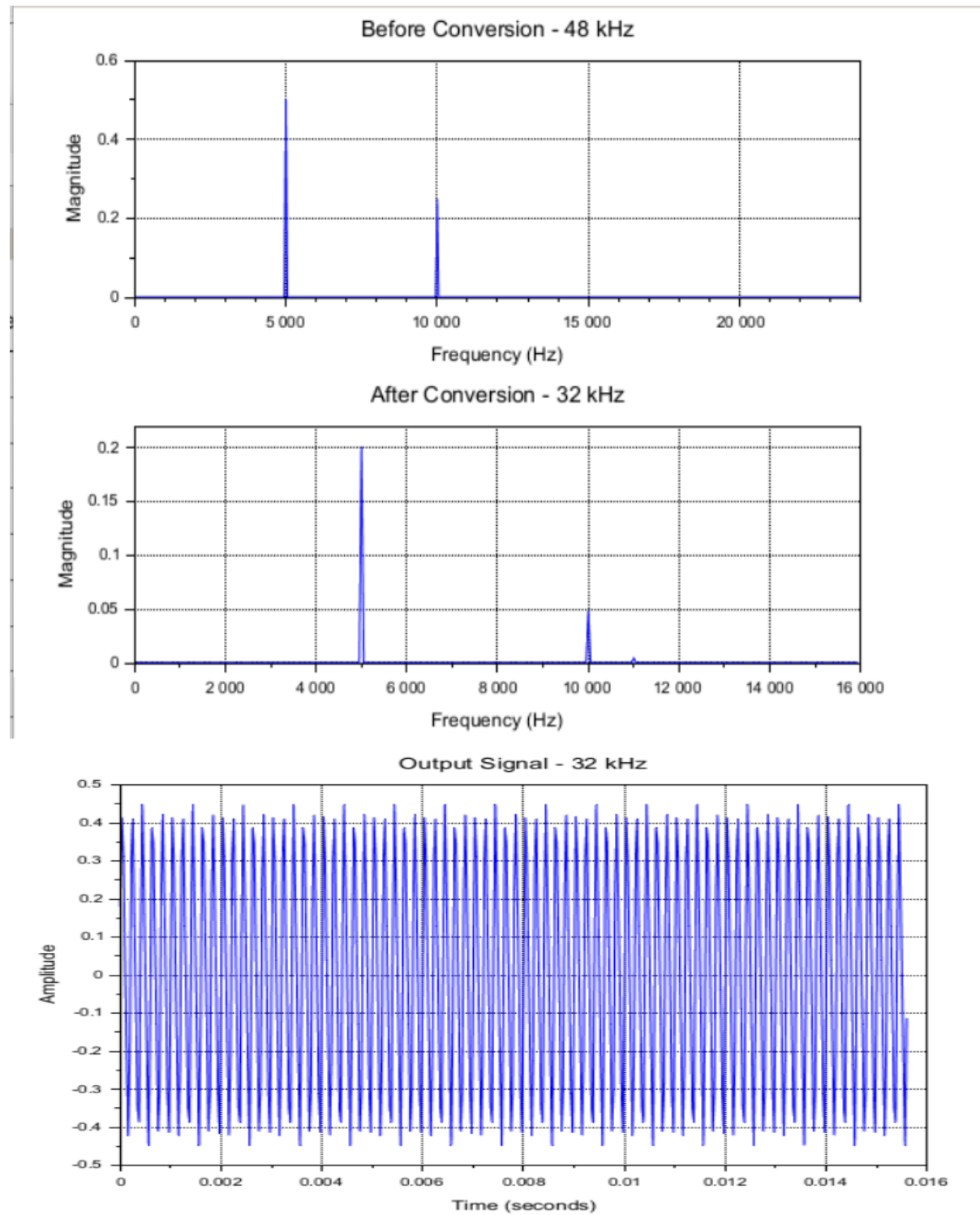
```

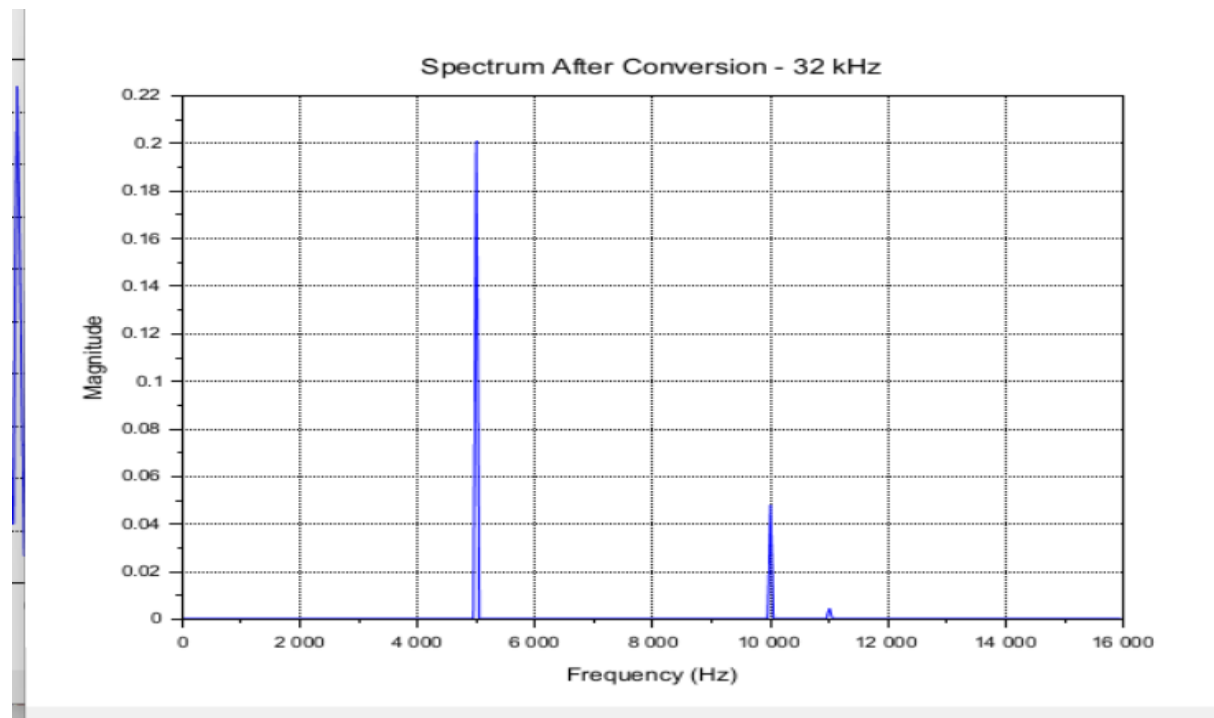
```

mprintf("Input Sampling Rate = %.2f kHz\n", ...
        Fs_in/1000);
mprintf("Interpolation Factor L = %d\n",L);
mprintf("Sampling Rate after Interpolation = %.2f kHz\n", ...
        Fs_interp/1000);
mprintf("Decimation Factor M = %d\n",M);
mprintf("Output Sampling Rate = %.2f kHz\n", ...
        Fs_final/1000);
mprintf("\nTheoretical Output Rate:\n");
mprintf("48 x (2/3) = %.2f kHz\n", ...
        Fs_in*(L/M)/1000);
// 11. Distortion Analysis
// Calculate signal power
signal_power = mean(x.^2);
// Calculate approximate output power
output_power = mean(x_out.^2);
// Display power values
mprintf("\n=====\n");
mprintf("DISTORTION ANALYSIS\n");
mprintf("=====\n");
mprintf("Input Signal Power = %f\n",signal_power);
mprintf("Output Signal Power = %f\n",output_power);
mprintf("\nLow-pass filtering reduces unwanted\n");
mprintf("spectral images and prevents aliasing.\n");

```

OUPTUT:





after interpolation = 96.00 kHz
after decimation = 32.00 kHz

=====

SAMPLING RATE CONVERSION

=====

Input Sampling Rate = 48.00 kHz
Interpolation Factor L = 2
Sampling Rate after Interpolation = 96.00 kHz
Decimation Factor M = 3
Output Sampling Rate = 32.00 kHz

Theoretical Output Rate:
 $48 \times (2/3) = 32.00 \text{ kHz}$

=====

DISTORTION ANALYSIS

=====

Input Signal Power = 0.625000
Output Signal Power = 0.085345

Low-pass filtering reduces unwanted
spectral images and prevents aliasing.

--->